

RK3562_NPU 使用说明



主题	NPU 使用说明
文档号	1.0
创建时间	2025-04-18
最后修改	2025-04-18
版本号	1.0
文件名	RK3562_NPU 使用说明 V1.0.pdf
文件格式	Portable Document Format

阅前须知

声明

北京合众恒跃科技有限公司保留随时对其产品进行修正、改进和完善的权利，客户在下单前应获取相关信息的最新版本，并验证这些信息是正确的。本文档一切解释权归北京合众恒跃科技有限公司所有。

简介

北京合众恒跃科技有限公司位于中关村科技园区，公司主要从事嵌入式产品的设计、研发、生产、销售等业务；公司的核心竞争力是对嵌入式产品精准的设计及实现能力，成本控制能力和产品品质保障能力；公司主干研发人员均有多年嵌入式产品开发经验和软硬件技术积累，服务于音视频、图像识别、电力等多个应用领域，公司研发设计的视频转码卡、图像识别卡、电力保护装置等产品在业内享有一定的知名度。

联系方式

北京合众恒跃科技有限公司

电话：010-62129511

邮编：102208

技术支持邮箱：support@hzhytech.com

地址：北京市海淀区安宁庄后街南 1 号 A 区一层 1020 号



官方微信公众号



官方企业微信



合众嵌入式官方店铺



合众 AI 官方店铺

修改记录

版本号	日期	修改人	备注
1.0	2025-04-18	马文轩	初始版本

目录

阅前须知	2
声明	2
简介	2
联系方式	2
修改记录	3
目录	4
第 1 章 NPU 使用说明	5
1.1 导入编译环境	5
1.1.1 创建工作目录并拷贝文件	5
1.1.2 解压编译链和 RKNN 数据包	5
1.1.3 导入 Docker 镜像	6
1.2 RKNN 编译	6
1.2.1 运行 Docker 容器	7
1.2.2 进入 RKNN 数据包目录并编译 YOLOv5 案例	8
1.2.3 查看编译输出	8
1.3 板卡端部署 RKNN	9
1.3.1 远程拷贝文件到板卡	9
1.3.2 登录板卡并准备环境	10
1.3.3 运行 YOLOv5 案例	10
1.3.4 获取并查看 NPU 运算结果	10

第 1 章 NPU 使用说明

此文档面向具有一定基础的用户，介绍如何快速在计算机上使用 RKNN-Toolkit2 完成模型转换，并通过 RKNPU2 部署到 Rockchip 开发板上。本文所用示例已集成到 RKNN Model Zoo 中。

支持的平台：RK3562、RK3566 系列、RK3568 系列、RK3576 系列、RK3588 系列。Docker 的使用参考《Docker 容器应用》。

1.1 导入编译环境

以下步骤详细介绍了如何导入和设置 SDK 的编译环境，包括创建工作目录、解压必要的文件以及导入 Docker 镜像。请根据实际需要替换版本号和其他具体信息。

1.1.1 创建工作目录并拷贝文件

- 1) 执行下面命令创建工作目录

```
mkdir RknnProject
```

- 2) 拷贝所有必要文件到工作目录

- Docker 镜像包
- RKNN 数据包
- GCC 交叉编译链

RKNN 工具包和 Docker 的版本号会根据后续更新而不断叠加，用户导入前需要仔细确定版本号，并在后续命令中替换为对应的版本号。

```
hzh@ubuntu:~/RknnProject$ ls -l
total 3828128
-rw-r----- 1 root root 223062142 May  9 10:43 gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu.tar.gz
-rw-r----- 1 root root 3473974272 May  9 10:44 hz-rknn-docker-v2.tar
-rw-r----- 1 root root 222949796 May  9 10:45 rknn_model_zoo-v2.3.0-2024-11-08.tgz
hzh@ubuntu:~/RknnProject$
```

图 1.1-1 查看版本号

1.1.2 解压编译链和 RKNN 数据包

- 执行下面命令解压 GCC 交叉编译链

```
tar -xf gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu.tar.gz
```

- 执行下面命令解压 RKNN 数据包

```
tar -xf rknn_model_zoo-v2.3.0-2024-11-08.tgz
```

解压后得到编译链文件夹和 RKNN 数据包文件夹。

```
hzhhy@ubuntu:~/RknnProject$ ls -la
total 3828144
drwxrwxr-x  4 hzhhy hzhhy    4096 May  9 10:49 .
drwxr-x---  5 hzhhy hzhhy    4096 May  9 10:43 ..
drwxrwxr-x 10 hzhhy hzhhy    4096 Feb 11 18:31 gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu
-rw-r-----  1 hzhhy hzhhy 223062142 May  9 10:43 gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu.tar.gz
-rw-r-----  1 hzhhy hzhhy 3473974272 May  9 10:44 hz-rknn-docker-V2.tar
drwx-----  9 hzhhy hzhhy    4096 Nov  8 20:30 rknn_model_zoo-v2.3.0-2024-11-08
-rw-r-----  1 hzhhy hzhhy 222949796 May  9 10:45 rknn_model_zoo-v2.3.0-2024-11-08.tgz
hzhhy@ubuntu:~/RknnProject$
```

图 1.1-2 查看编译后的文件夹及 RKNN 数据包文件夹

1.1.3 导入 Docker 镜像

- 执行下面命令导入 RKNN 镜像

```
docker load -i hz-rknn-docker-V2.tar
```

- 执行下面命令查看导入的镜像

```
docker images
```

```
hzhhy@ubuntu:~/RknnProject$ sudo docker load -i hz-rknn-docker-V2.tar
Loaded image: hz-rknn-docker:V2
hzhhy@ubuntu:~/RknnProject$
hzhhy@ubuntu:~/RknnProject$ sudo docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
hz-rknn-docker-u24   V3                 aceb1a26d14d       2 weeks ago        855MB
hz-rknn-docker       V2                 8f5b198f3256       2 weeks ago        3.44GB
ubuntu               24.04              602eb6fb314b       4 weeks ago        78.1MB
ubuntu               22.04              cc934a90cd99       4 weeks ago        77.9MB
hello-world          latest              74cc54e27dc4       3 months ago       10.1kB
rknn-toolkit2        2.3.0-cp38         40ca8bd2889c       6 months ago       3.33GB
hzhhy@ubuntu:~/RknnProject$
```

图 1.1-3 查看导入 Docker 镜像

注意事项:

- 1) **版本号确认:** RKNN 工具包和 Docker 镜像的版本号会随更新而变化, 请在导入前仔细确认所需版本, 并在命令中替换为对应的版本号。
- 2) **路径与名称:** 确保文件名和路径正确无误, 避免因拼写错误导致的操作失败。

1.2 RKNN 编译

导入完成后，运行 Docker 容器，并将所需要的目录映射到容器内。执行命令后将自动运行容器，并进入容器交互式命令行。

1.2.1 运行 Docker 容器

执行以下命令来运行 Docker 容器，并将所需的目录映射到容器内。

```
docker run -it --privileged \  
-v /dev/bus/usb:/dev/bus/usb \  
-v $(pwd)/rknn_model_zoo-v2.3.0-2024-11-08/rknn_model_zoo \  
-v $(pwd)/gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu:/arm-gcc \  
hz-rknn-docker:V1 /bin/bash
```

参数说明：

- `-it`：以交互模式启动容器。
- `--privileged`：赋予容器扩展权限，以便访问 USB 设备等资源。
- `-v /dev/bus/usb:/dev/bus/usb`：将主机的 USB 设备挂载到容器中。
- `-v $(pwd)/rknn_model_zoo-v2.3.0-2024-11-08/rknn_model_zoo`：将主机上的 RKNN 数据包目录挂载到容器中的 `/rknn_model_zoo` 路径。
- `-v $(pwd)/gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu:/arm-gcc`：将主机上的 GCC 交叉编译链目录挂载到容器中的 `/arm-gcc` 路径。
- `hz-rknn-docker:V1`：使用的 Docker 镜像名称及标签。
- `/bin/bash`：进入容器的交互式命令行。

```
root@bd4d82c95031:/#  
root@bd4d82c95031:/# ls -l  
total 39428  
drwxrwxr-x 10 1002 1002 4096 Feb 11 10:31 arm-gcc  
lrwxrwxrwx 1 root root 7 Jun 5 2023 bin -> usr/bin  
drwxr-xr-x 2 root root 4096 Apr 15 2020 boot  
drwxr-xr-x 15 root root 3920 Apr 18 07:06 dev  
drwxr-xr-x 1 root root 4096 Apr 18 07:06 etc  
drwxr-xr-x 2 root root 4096 Apr 15 2020 home  
lrwxrwxrwx 1 root root 7 Jun 5 2023 lib -> usr/lib  
lrwxrwxrwx 1 root root 9 Jun 5 2023 lib32 -> usr/lib32  
lrwxrwxrwx 1 root root 9 Jun 5 2023 lib64 -> usr/lib64  
lrwxrwxrwx 1 root root 10 Jun 5 2023 libx32 -> usr/libx32  
drwxr-xr-x 2 root root 4096 Jun 5 2023 media  
drwxr-xr-x 2 root root 4096 Jun 5 2023 mnt  
drwxr-xr-x 2 root root 4096 Jun 5 2023 opt  
dr-xr-xr-x 704 root root 0 Apr 18 07:06 proc  
drwx----- 11 1002 1002 4096 Apr 18 07:24 rknn_model_zoo  
-rw-r--r-- 1 root root 40303964 Nov 7 06:15 rknn_toolkit2-2.3.0-cp38-cp38-manylinux_2_17_x86_64.manylinux2014_x86_64.whl  
drwx----- 1 root root 4096 Apr 18 07:24 root  
drwxr-xr-x 1 root root 4096 Apr 18 03:50 run  
lrwxrwxrwx 1 root root 8 Jun 5 2023 sbin -> usr/sbin  
drwxr-xr-x 2 root root 4096 Jun 5 2023 srv  
dr-xr-xr-x 13 root root 0 Apr 18 07:06 sys  
drwxrwxrwt 1 root root 4096 Apr 18 07:34 tmp  
drwxr-xr-x 1 root root 4096 Jun 5 2023 usr  
drwxr-xr-x 1 root root 4096 Jun 5 2023 var  
root@bd4d82c95031:/#  
root@bd4d82c95031:/#
```

图 1.2-1 运行 Docker 容器

1.2.2 进入 RKNN 数据包目录并编译 YOLOv5 案例

- 1) 执行下面命令进入 RKNN 数据包目录

```
cd rknn_model_zoo
```

- 2) 编译 YOLOv5 案例

这里使用 YOLOv5 案例进行验证测试，其他案例用户可以自行编译验证。执行 YOLOv5 编译命令。

参数说明：

- -t rk356x: 目标平台为 RK356x 系列
- -a aarch64: 架构为 AArch64
- -d yolov5: 选择 YOLOv5 作为示例模型

```
./build-linux.sh -t rk356x -a aarch64 -d yolov5
```

```
root@bd4d82c95031:/rknn_model_zoo# ./build-linux.sh -t rk356x -a aarch64 -d yolov5
./build-linux.sh -t rk356x -a aarch64 -d yolov5
/arm-gcc/bin/aarch64-none-linux-gnu-g++
=====
BUILD_DEMO_NAME=yolov5
BUILD_DEMO_PATH=examples/yolov5/cpp
TARGET_SOC=rk356x
TARGET_ARCH=aarch64
BUILD_TYPE=Release
ENABLE_ASAN=OFF
DISABLE_RGA=OFF
INSTALL_DIR=/rknn_model_zoo/install/rk356x_linux_aarch64/rknn_yolov5_demo
BUILD_DIR=/rknn_model_zoo/build/build_rknn_yolov5_demo_rk356x_linux_aarch64_Release
CC=/arm-gcc/bin/aarch64-none-linux-gnu-gcc
CXX=/arm-gcc/bin/aarch64-none-linux-gnu-g++
=====
-- Configuring done
-- Generating done
-- Build files have been written to: /rknn_model_zoo/build/build_rknn_yolov5_demo_rk356x_linux_aarch64_Release
[ 16%] Built target fileutils
[ 33%] Built target imagedrawing
[ 50%] Built target audiotools
[ 66%] Built target imageutils
[100%] Built target rknn_yolov5_demo
[ 16%] Built target fileutils
[ 33%] Built target imageutils
[ 50%] Built target imagedrawing
[ 83%] Built target rknn_yolov5_demo
[100%] Built target audiotools
Install the project...
-- Install configuration: "Release"
-- Installing: /rknn_model_zoo/install/rk356x_linux_aarch64/rknn_yolov5_demo/.rknn_yolov5_demo
-- Set runtime path of "/rknn_model_zoo/install/rk356x_linux_aarch64/rknn_yolov5_demo/.rknn_yolov5_demo" to "$ORIGIN/lib"
-- Installing: /rknn_model_zoo/install/rk356x_linux_aarch64/rknn_yolov5_demo/.model/bus.jpg
-- Installing: /rknn_model_zoo/install/rk356x_linux_aarch64/rknn_yolov5_demo/.model/coco_80_labels_list.txt
-- Installing: /rknn_model_zoo/install/rk356x_linux_aarch64/rknn_yolov5_demo/model/yolov5s_relu.rknn
-- Installing: /rknn_model_zoo/install/rk356x_linux_aarch64/rknn_yolov5_demo/lib/librknnrt.so
-- Installing: /rknn_model_zoo/install/rk356x_linux_aarch64/rknn_yolov5_demo/lib/librga.so
```

图 1.2-2 编译

1.2.3 查看编译输出

- 1) 进入编译输出目录

编译完成后执行下面命令，进入编译输出目录。

```
cd install/rk356x_linux_aarch64
```

- 2) 查看编译文件

使用下面命令查看编译文件。可以看到编译出来的库文件和可执行文件。

```
tree rknn_yolov5_demo
```

```
root@bd4d82c95031:/rknn_model_zoo# cd install/rk356x_linux_aarch64/
root@bd4d82c95031:/rknn_model_zoo/install/rk356x_linux_aarch64#
root@bd4d82c95031:/rknn_model_zoo/install/rk356x_linux_aarch64# ls
rknn_yolov5_demo
root@bd4d82c95031:/rknn_model_zoo/install/rk356x_linux_aarch64#
root@bd4d82c95031:/rknn_model_zoo/install/rk356x_linux_aarch64# tree rknn_yolov5_demo/
rknn_yolov5_demo/
|-- lib
|   |-- librga.so
|   |-- librknnrt.so
|-- model
|   |-- bus.jpg
|   |-- coco_80_labels_list.txt
|   |-- yolov5s_relu.rknn
|-- rknn_yolov5_demo

2 directories, 6 files
root@bd4d82c95031:/rknn_model_zoo/install/rk356x_linux_aarch64#
```

图 1.2-3 查看编译结果

注意事项：

- 1) 版本号确认：确保使用的 RKNN 工具包和 Docker 镜像版本号正确无误。
- 2) 路径与名称：检查所有路径和文件名是否正确，避免因拼写错误导致的操作失败。
- 3) 依赖项安装：如果在编译过程中遇到缺少依赖项的问题，请根据提示安装相应的软件包。

1.3 板卡端部署 RKNN

在完成编译后，下一步是将生成的文件和库拷贝到目标板卡上，并执行 YOLOv5 案例进行 NPU 运算。

以下是详细的步骤和命令，帮助您顺利完成这些操作。

1.3.1 远程拷贝文件到板卡

- 1) 拷贝库文件到板卡

执行远程拷贝命令将所有文件拷贝到板卡上，这里需要根据板卡实际 IP 和位置拷贝。

```
root@bd4d82c95031:/rknn_model_zoo/install/rk356x_linux_aarch64# scp -r rknn_yolov5_demo/ root@192.168.1.163:~
root@192.168.1.163's password:
rknn_yolov5_demo
librknnrt.so
librga.so
coco_80_labels_list.txt
bus.jpg
yolov5s_relu.rknn
root@bd4d82c95031:/rknn_model_zoo/install/rk356x_linux_aarch64#
```

100%	964KB	17.2MB/s	00:00
100%	7089KB	24.1MB/s	00:00
100%	192KB	18.1MB/s	00:00
100%	621	587.9KB/s	00:00
100%	177KB	18.5MB/s	00:00
100%	7473KB	25.0MB/s	00:00

图 1.3-1 拷贝文件

- 2) 拷贝整个 YOLOv5 案例目录到板卡

执行下面命令将案例中库拷贝到板卡系统中。

```
cp rknn_yolov5_demo/lib/* /lib
```

1.3.2 登录板卡并准备环境

1) 登录板卡

使用 SSH 登录到板卡。

2) 进入 YOLOv5 案例目录

执行下面命令进入案例目录，执行 `./rknn_yolov5_demo ./model/yolov5s_relu.rknn ./model/bus.jpg` 开始 NPU 运算。

```
cd rknn_yolov5_demo/
```

1.3.3 运行 YOLOv5 案例

在板卡上运行 YOLOv5 示例程序。

```
./rknn_yolov5_demo ./model/yolov5s_relu.rknn ./model/bus.jpg
```

```
root@rk3562-buildroot:~/rknn_yolov5_demo#  
root@rk3562-buildroot:~/rknn_yolov5_demo# ./rknn_yolov5_demo ./model/yolov5s_relu.rknn ./model/bus.jpg  
load label ./model/coco_80_labels_list.txt  
model input num: 1, output num: 3  
input tensors:  
  index=0, name=images, n_dims=4, dims=[1, 640, 640, 3], n_elems=1228800, size=1228800, fmt=NHWC, type=INT8, qnt_type=AFFINE, zp=-128, scale=0.003922  
output tensors:  
  index=0, name=output0, n_dims=4, dims=[1, 255, 80, 80], n_elems=1632000, size=1632000, fmt=NCHW, type=INT8, qnt_type=AFFINE, zp=-128, scale=0.003922  
  index=1, name=286, n_dims=4, dims=[1, 255, 40, 40], n_elems=408000, size=408000, fmt=NCHW, type=INT8, qnt_type=AFFINE, zp=-128, scale=0.003922  
  index=2, name=288, n_dims=4, dims=[1, 255, 20, 20], n_elems=102000, size=102000, fmt=NCHW, type=INT8, qnt_type=AFFINE, zp=-128, scale=0.003922  
model is NHWC input fmt  
model input height=640, width=640, channel=3  
origin size=640x640 crop size=640x640  
input image: 640 x 640, subsampling: 4:2:0, colorspace: YCbCr, orientation: 1  
scale=1.000000 dst_box=(0 0 639 639) allow_slight_change=1 _left_offset=0 _top_offset=0 padding_w=0 padding_h=0  
rga_api version 1.10.1 [0]  
rknn_run  
person @ (200 243 286 510) 0.880  
person @ (470 238 560 526) 0.871  
person @ (100 238 231 534) 0.840  
bus @ (91 120 555 464) 0.692  
person @ (79 353 121 517) 0.301  
write_image path: out.png width=640 height=640 channel=3 data=0xdf73e90  
root@rk3562-buildroot:~/rknn_yolov5_demo#
```

图 1.3-2 运行 YOLOv5 示例程序

这条命令会加载模型 `yolov5s_relu.rknn` 并对图像 `bus.jpg` 进行推理，输出结果保存为 `out.png`。

1.3.4 获取并查看 NPU 运算结果

1) 获取输出图片

将生成的 `out.png` 从板卡拷贝回 PC 主机。

2) 查看结果

在 PC 主机上打开 `out.png`，查看 NPU 运算的结果。

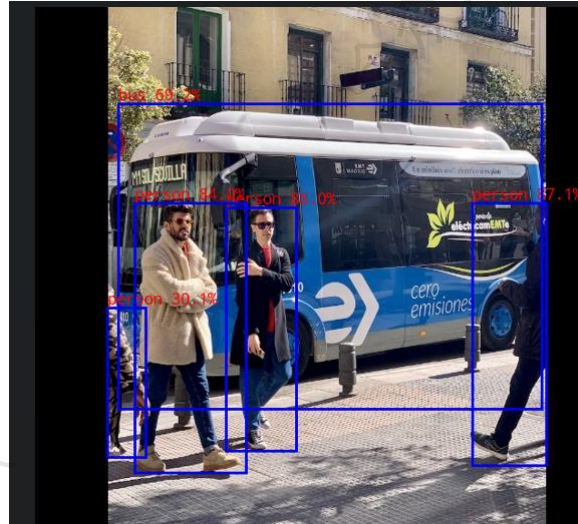


图 1.3-3 查看 otu.png